

Report: Predicting City Attractiveness

Project canvas reflection

We initially submitted two project canvases: one about benchmarking various implementations of the QUIC protocol (Google's (practically) replacement for TCP, on web but really elsewhere too), and one about finding ways to eliminate congestion points in city traffic. Two, because we were unsure of both of them, and turns out maybe we were right to be a bit unsure.

Initially we chose to go with the QUIC idea and did in fact even set up a server abroad (one from a friend in Austria and one cheap (but networking-wise powerful) server in Amsterdam) and got a bunch of data using [Quinn's](#) built-in benchmarking logic (adapted to our code). However, Quinn alone took some very serious effort to set up and it was about to wind up being a mostly programming task rather than having much to do with data science, plus one of us isn't a programmer, so we decided to ditch this and try the traffic idea instead.

For the traffic idea, we first wanted to try identify cases where [Braess' paradox](#) applies (that is, cases where an additional road is actually *slowing down* traffic instead of improving it, despite offering a factually shorter route). This we ended up ditching very quickly however, mainly for two reasons: We didn't find accurate enough traffic data to identify such cases (data was available only for some roads, and didn't find a good alternative; we looked into Google's Maps API and even considered using air quality data from FMI, but weren't convinced we could actually execute our original plan). We also got feedback from the TA that the focus was off for the course, and after brainstorming some alternative ideas related to traffic data, we ended up finding a completely unrelated idea that we ultimately went with.

That idea being this project; mapping of city (municipality, in practice) properties (f.e population count and housing prices) to an 'attractiveness' value, where attractiveness is a measure of roughly whether the municipality population is increasing or decreasing. This was all about data with some supposedly simple programming we'd already learned on the course (though the programming parts ended up being very difficult and time-consuming nonetheless), and got TA's approval, and ended up being feasible.

Possible future steps

There's a lot that could've been done with a lot more time: perhaps the most important factor in the accuracy of the model is what properties we choose to use, and there's a lot of work to do in that regard. It's not obvious (and shouldn't assume it to be) which properties have how much correlation, and we didn't get to touch this much at all, so this would be the by far biggest thing to dive into next -- to actually spend time just finding properties that correlate really well. (We did find correlation in our chosen datasets too, so I'm sure better ones exist.)

Next, this could've surely been wired up to a website where you can select a municipality and interactively either try adjusting the properties, or to have a tool that automatically identifies the

order of importance of the properties of each municipality, for example. This would've made the project vastly more actually usable. I suppose we ran both out of time and out of steam to get this far.

Also: the model used is a linear regression model from Sci-kit, but that isn't to say it's the best model for the purpose of course, so one could also try different models (or [TPOT](#) to automate the task) to see if that makes a big difference. That said, all these aside, we were getting a score of 0.97 with our setup, which while not actually as good as it sounds judging by the score, was actually okay in practice, too.

And most unfortunately of all, we ended up making pretty poor use of our datasets in the sense that each of them contained data for many years, but we only ended up using the last year's data.

Technical details

The setup ended up being quite simple for how much time it took:

1. We fetch per-municipality population statistics and use that to estimate the attractiveness of each city (since attractiveness was chosen to be the increase/decrease of population, this is a perfect fit). As mentioned, we unfortunately only ended up using the last year's data, but it does provide an accurate view of the last year's population change of course, so more data isn't more accurate, it's just that more data would've given us more training data, too, to better train the model.
2. We then wrangle the data of the chosen properties of each municipality into one big Pandas dataframe. This probably took the most time because of our unfamiliarity with Pandas.
3. Using the population change-based attractiveness as target and the dataframe holding the properties data as inputs, we train the linear regression model.
4. Visualized using GeoPandas.

Overall experience & reflection

Difficulties: We were very new to data science, not familiar at all with tools like Pandas and lacked theoretical background for data learning. Learning this took the most time and limited our ability to develop the project properly.

In retrospect, we now feel that we chose our project idea to force the use of machine learning, not to study the relationships in the data. Studying the relationship between the city features and their attractiveness, now seems like a better project idea. But we also wanted a "tool" as our project's outcome, so we didn't think to consider an analysis project like that.

Every word written without use of AI